Motor de Análisis Técnico de Acciones

Evaluación determinista + IA solo interpretativa

Rubén Touceda · Máster Desarrollo con IA (Big School) · Octubre 2026

El problema

- Filtraba acciones en Finviz por parámetros, pero luego tocaba ir **una a una** clasificándolas como más o menos interesantes
- Ese triaje manual no escala: decenas de candidatas pidiendo **análisis fundamental** (históricos, beneficios, ganancias)
- Faltaba un **prefiltro cuantitativo**: que cada acción se evalúe sola con ratio riesgo/beneficio + entrada + stop-loss…
- …para quedarme solo con las mejores y dedicarles el análisis fundamental

La idea: división clara

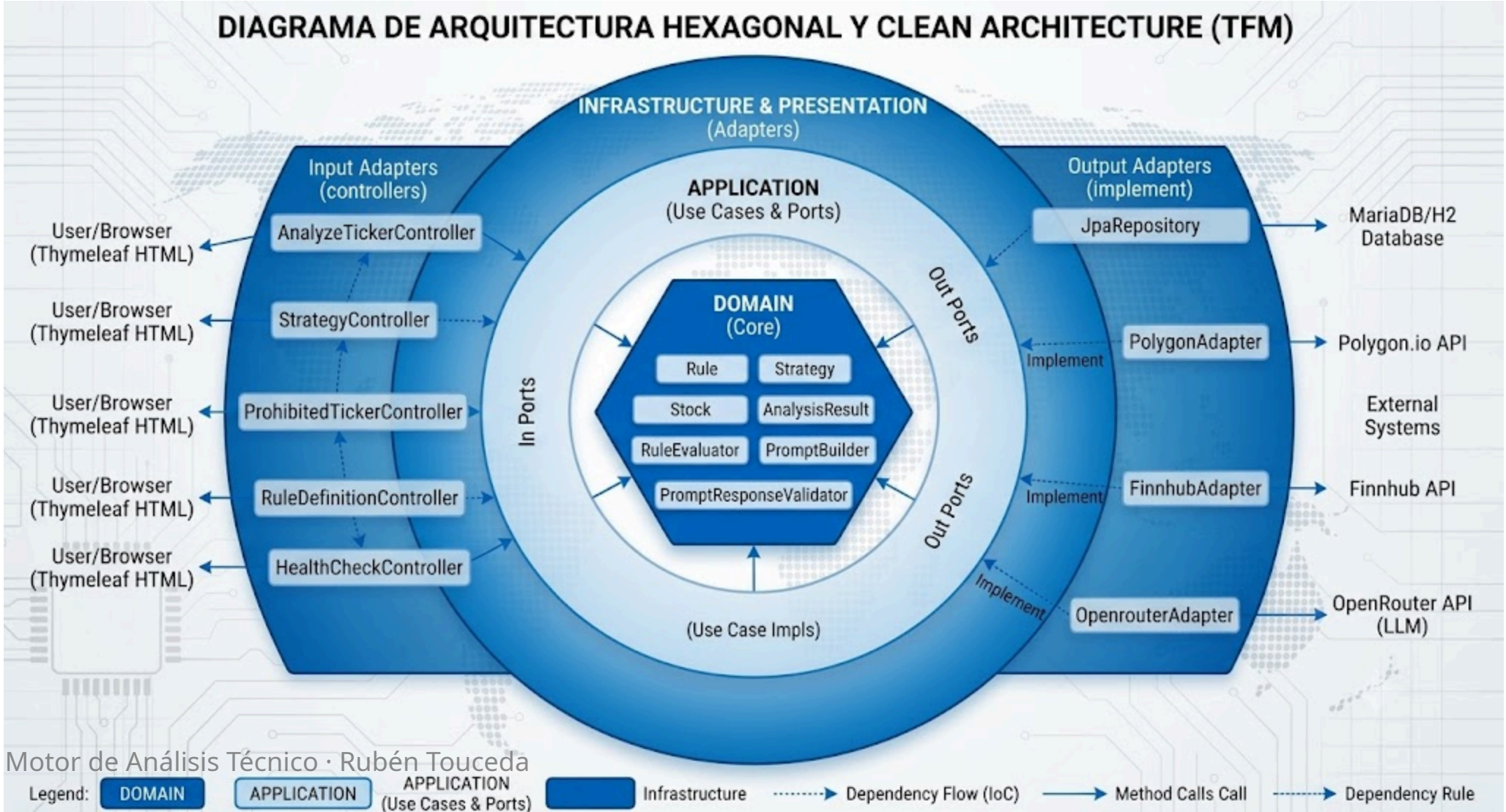
1. **Motor determinista**: reglas técnicas → score 0-100 + R:R + cumplimiento
2. **IA solo interpretativa**: redacta la síntesis desde métricas ya calculadas

Si la IA falla (429, vacíos), la evaluación numérica **sigue intacta** y la vista muestra un aviso amistoso con fallback.

¿Cómo funciona?

1. Ingesta de tickers o sugerencias (Finviz) con filtro de seguridad
2. Enriquecimiento: OHLCV + SMA 20/50/200 y volumen (Polygon/Finnhub)
3. Evaluación declarativa: `Precio > SMA(50)` AND `Volumen > Media(20)`
4. Cálculo de R:R y score ponderado 0-100 por estrategia
5. Análisis IA a petición (async, con rotación de modelos ante 429)
6. Persistencia y seguimiento

Arquitectura hexagonal



Motor de reglas + scoring

- `Rule`: condición autocontenida → booleano + justificación
- `Strategy`: composición AND + `threshold`; cada regla tiene `weight`
- Catálogo de capacidades: qué indicador acepta qué parámetros (rechaza `SMA(100)` antes de evaluar, sin fallos silenciosos)
- Todo queda auditado: qué regla falló y por qué

Fronteras de la IA (diseño intencionado)

Motor determinista	IA interpretativa
Evalúa reglas y métricas	Redacta síntesis desde métricas
Reproducible, testeable	No decide, no valida, no señaliza
1165 tests lo cubren	Si falla → fallback + aviso

Modelo principal `laguna-s-2.1` + 4 reservas · Planes gratuitos: latencias altas **asumidas por diseño**, importa la corrección, no la velocidad.

Stack y calidad

- **Backend**: Java 21 · Spring Boot · JPA · WebClient · Thymeleaf + Bootstrap 5
- **Datos**: H2 (dev) · MariaDB (prod) · Finnhub / Polygon / Finviz / OpenRouter
- **Calidad**: 1165 tests, JaCoCo ≥80%, **SonarQube A** (63→0 issues), CI en PRs
- **Despliegue**: Docker + VPS propio con Nginx/SSL → https://tfm.rubentouceda.es
- Se evaluó Railway, pero la comunicación con la base de datos era lenta; el VPS además amplió conocimientos (endurecimiento, fail2ban, backups, red y certificados gestionados a mano)

Desarrollo con IA: 3 fases

1. **Copilot**: arranque, estructura base, autocompletado
2. **OpenCode supervisado**: 1 rama = 1 tema = 1 doc = 1 PR, validación con menú, sin avanzar sin aprobación (~25 PRs, `docs/` con trazabilidad total)
3. **Desarrollo propio**: diseño, tuning, imprevistos de producción

Frontera sagrada: **la IA propone, la persona dispone**.

Demo (5 min)

En https://tfm.rubentouceda.es:

1. Login → `/analysis`: ingesta de un ticker y su evaluación con score
2. Detalle del ticker: velas + SMA + plan de riesgo
3. Generar análisis IA → banner con progreso → síntesis en 4 secciones
4. `/strategies`: crear estrategia y ver sugerencias

Limitaciones y futuro

Hoy no hace: recomendar inversión · operar · tiempo real · notificaciones
Líneas futuras:

- OpenAPI pública cuando haya API REST real
- Servicio ML separado (Python/FastAPI) desacoplado del motor
- Migración Spring Boot 4.x (fin del soporte OSS 3.5.x)

Cierre: las 5 piezas

- ✅ README que se entiende sin abrir código
- ✅ Código supervisado en GitHub
- ✅ Despliegue productivo funcionando
- ✅ Estas slides + speaker notes versionadas
- ⬜ Formulario final con las URLs

Gracias — preguntas.